Exploratory data analysis (EDA)

🎯 Today's goal

By the end of this hour you will:

- follow a step-by-step **EDA routine** to get to know a new dataset
- describe how values are spread with `describe()`, histograms and `value_counts(normalize=True)`
- compare groups with `groupby` and `pd.crosstab`
- measure how two columns move together with the **correlation** `.corr()`, and know why it doesn't prove cause
- find **outliers** with the IQR rule, and write your findings in plain English

🤔 Why this matters

Exploratory data analysis (EDA) means getting to know a dataset before you trust it or build anything on it. It's like a doctor's first examination: look, measure, compare, notice anything unusual, and only then decide what to do.

Every real data project starts this way, and so will the machine-learning days ahead. EDA uses everything from the last five days: pandas for loading, filtering and grouping, cleaning for the problems you find, and charts to see patterns. Today you'll put it all together into one routine you can reuse on any dataset.

🧭 The EDA routine

1. **Questions first:** what do you want to find out?
2. **Shape and types:** how many rows and columns, and what kind is each column?
3. **Missing values:** where are the gaps?
4. **Distributions:** how is each column spread out?
5. **Groups:** do categories differ from each other?
6. **Relationships:** do two columns move together?
7. **Outliers:** are there values that don't fit?
8. **Findings:** write down what you learned, in plain words.

🎒 Today's data: a student survey

Imagine a survey of 200 students from three schools: hours of study per week, hours of sleep per night, attendance (the percentage of classes they came to) and their exam score. The data is **made up**: we generate it with NumPy's random generator and a fixed seed, so everyone gets exactly the same numbers. We also planted a few problems for you to find. Don't worry about understanding every line of the generator; the point is the table it makes.

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (generated with a fixed seed)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)    # per week
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)      # per night
attendance = rng.normal(88, 7, n).clip(55, 100).round()     # % of classes
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()

students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
# Planted problems: 9 missing sleep values and 3 suspicious study hours
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

print(students.shape)
students.head()
```

```
(200, 5)
```

	school	study_hours	sleep_hours	attendance	score
0	East	11.3	7.8	84.0	75.0
1	South	7.6	7.2	94.0	66.0
2	East	9.0	7.5	90.0	69.0
3	South	8.4	6.3	76.0	53.0
4	North	10.2	6.8	92.0	63.0

❓ Steps 1–3: questions, shape and gaps

Questions first. Without questions you'll wander around the data. Ours:

1. Do students who study more get higher scores? What about sleep and attendance?
2. Do the three schools differ?
3. Is anything in the data wrong or strange?

The output above already told us the **shape**: 200 rows (students) and 5 columns. Now the **types** and the **gaps**:

```
print(students.dtypes)
print()
print("Missing values per column:")
print(students.isna().sum())
```

```
school            str
study_hours     float64
sleep_hours     float64
attendance      float64
score           float64
dtype: object

Missing values per column:
school          0
study_hours     0
sleep_hours     9
attendance      0
score           0
dtype: int64
```

One text column (`school`), four number columns, and 9 students without a sleep value. That's 4.5% of the rows: few enough that most calculations can simply skip them (pandas does that automatically). Write it down for your findings.

📶 Step 4: distributions

A **distribution** is how the values of a column are spread out: where most of them are, how far they reach, and whether some are far away from the rest. `describe()` sums it up in numbers:

```
students.describe().round(1)
```

	study_hours	sleep_hours	attendance	score
count	200.0	191.0	200.0	200.0
mean	9.6	7.0	87.5	61.5
std	5.3	1.0	6.9	12.5
min	0.0	4.0	70.0	33.0
25%	6.8	6.3	82.0	53.0
50%	9.2	6.9	88.0	62.0
75%	11.5	7.6	92.0	70.0
max	45.0	9.4	100.0	93.0

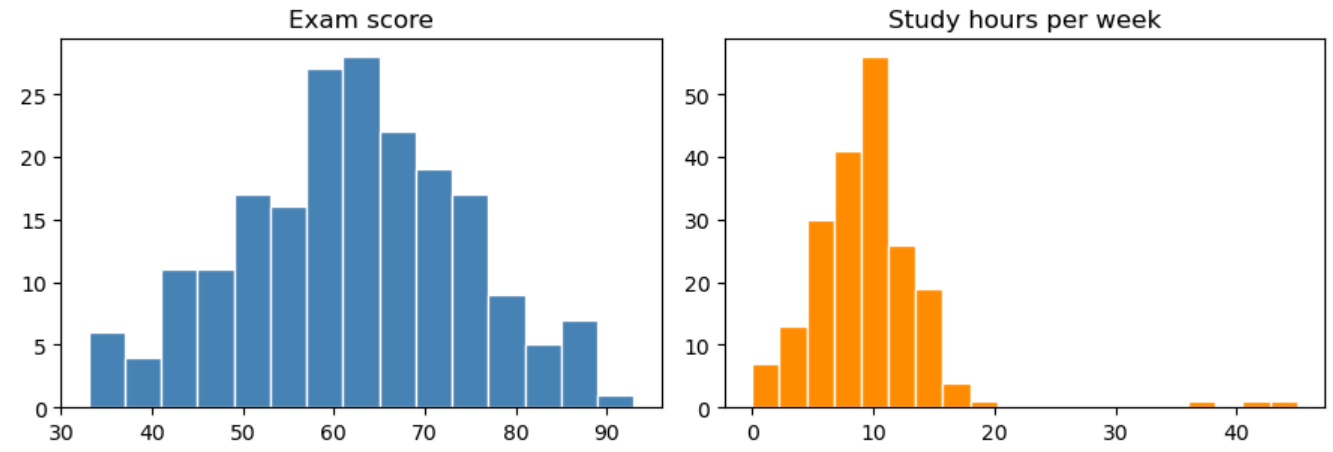
How to read it, row by row:

- **count:** filled cells (sleep has only 191).
- **mean:** the average. **std** (*standard deviation*): how far values typically are from the mean.
- **min / max:** the smallest and largest value. Look here first for strange values!
- **25%, 50%, 75%:** the *quartiles*. A quarter of the students are below the 25% value, half are below the 50% value (the *median*), three quarters are below the 75% value.

Did you spot it? Most students study 7 to 12 hours a week, but the **max is 45**. That's suspicious. Keep it in mind. A histogram shows the shapes:

```
import matplotlib.pyplot as plt

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.2))
left.hist(students["score"], bins=15, color="steelblue", edgecolor="white")
left.set_title("Exam score")
right.hist(students["study_hours"], bins=20, color="darkorange",
           edgecolor="white")
right.set_title("Study hours per week")
plt.tight_layout()
plt.show()
```



Scores form a hill around 60. Study hours form a hill around 9, plus a few lonely bars far to the right: those are the strange values. For a text column, the "distribution" is simply how often each value appears. `normalize=True` gives **shares** (fractions of 1) instead of counts:

```
students["school"].value_counts(normalize=True)
```

```
school
North    0.395
South    0.360
East     0.245
Name: proportion, dtype: float64
```

So about 40% of the students are from North, 36% from South and 25% from East.

🏫 Step 5: comparing groups

Do the schools differ? `groupby` from Day 11 answers that:

```
columns = ["score", "study_hours", "sleep_hours"]
by_school = students.groupby("school")[columns].mean()
by_school.round(1)
```

	score	study_hours	sleep_hours
school			
East	62.0	9.7	7.0
North	62.9	9.1	6.9
South	59.7	10.1	7.0

North has the highest average score, but the differences are only a few points. With about 50 to 80 students per school, small differences like these could easily be luck. Be careful not to over-sell them.

A **crosstab** (cross-tabulation) counts how often each combination of two categories appears. Let's add a `passed` column (score of 60 or more) and cross it with the school. `normalize="index"` turns each row into shares, so each row adds up to 1:

```
students["passed"] = students["score"] >= 60

counts = pd.crosstab(students["school"], students["passed"])
shares = pd.crosstab(students["school"], students["passed"], normalize="index")
print(counts)
print()
print(shares.round(2))
```

```
passed  False  True
school
East       18     31
North      33     46
South      35     37

passed  False  True
school
East     0.37   0.63
North    0.42   0.58
South    0.49   0.51
```

Interesting: East has the highest *pass rate* (63%) even though North has the highest *average*. Different measures can tell slightly different stories, which is exactly why we look at more than

one.

🔗 Step 6: relationships and correlation

Do students who study more score higher? The **correlation**, written *r*, measures how strongly two number columns move together in a straight-line way. It is always between −1 and +1:

r	Meaning
close to +1	when one goes up, the other goes up too (strong)
around +0.3 to +0.7	they tend to go up together, with plenty of exceptions
close to 0	no straight-line relationship
negative	when one goes up, the other tends to go down

Correlation only works on numbers, so calling it on the whole table fails:

💡 This example has a mistake on purpose. Run it and read the error.

```
students.corr()
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     students.corr()
>     ~~~~~~~~~~~~~^^
> ValueError: could not convert string to float: 'East'
> ```

"could not convert string to float: 'East'": pandas tried to turn the school names into numbers. Pick the number columns first. For one pair of columns use `column_a.corr(column_b)`:

```
number_columns = ["study_hours", "sleep_hours", "attendance", "score"]
print(students[number_columns].corr().round(2))
print()

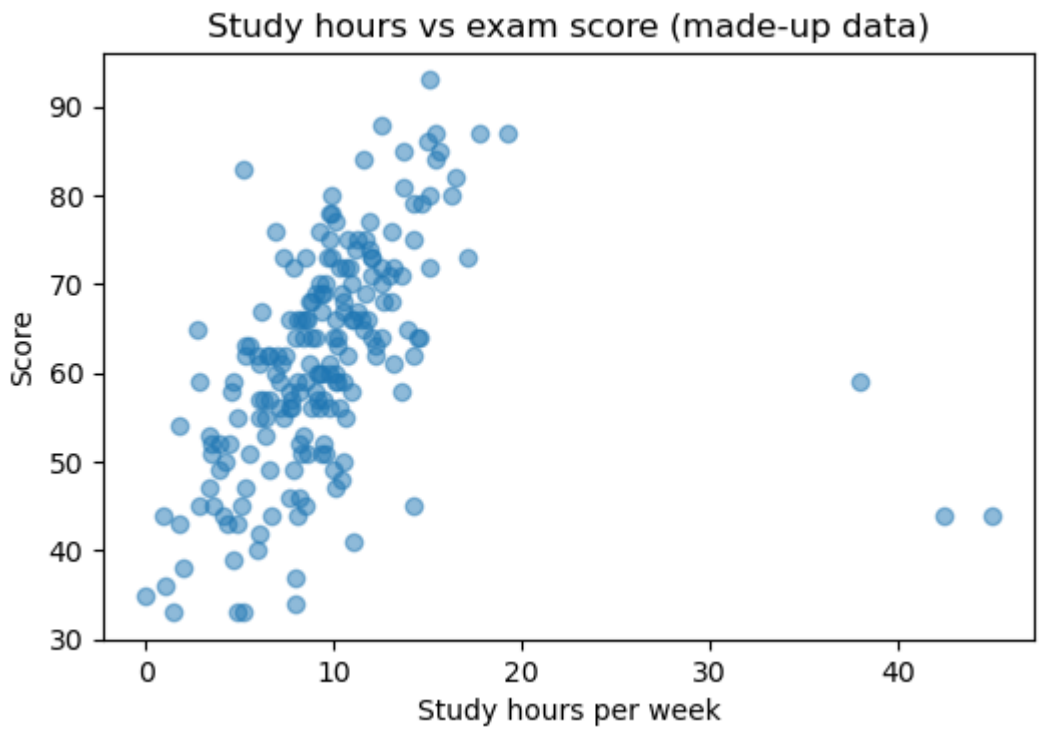
r = students["study_hours"].corr(students["score"])
print(f"Study hours vs score: r = {r:.2f}")
```

```
             study_hours  sleep_hours  attendance  score
study_hours         1.00         0.04        0.00   0.35
sleep_hours         0.04         1.00        0.07   0.20
attendance          0.00         0.07        1.00   0.15
score               0.35         0.20        0.15   1.00

Study hours vs score: r = 0.35
```

Read the table by finding a row and a column: where *score* meets *study_hours* you find their r. (The diagonal is always 1: every column matches itself perfectly.) Study hours have the strongest link with the score, about 0.35. A scatter plot shows what's behind that number:

```
plt.figure(figsize=(6, 3.8))
plt.scatter(students["study_hours"], students["score"], alpha=0.5)
plt.title("Study hours vs exam score (made-up data)")
plt.xlabel("Study hours per week")
plt.ylabel("Score")
plt.show()
```



A clear upward cloud on the left, and three lonely dots far to the right. Those three points pull the line and make r smaller than it should be.

Correlation is not causation. Even a strong r only says two things go together, not that one *causes* the other. Ice-cream sales and sunburns rise together, but ice cream doesn't cause sunburn: hot sunny weather causes both. Maybe students who study more are also more motivated, and motivation raises the score. Data like ours can't tell the difference.

🚩 Step 7: outliers

An **outlier** is a value far away from the rest. Some are mistakes (a typo of 38 instead of 3.8), some are real but rare. A common rule to flag them uses the **IQR** (*interquartile range*): the distance between the 25% value (Q1) and the 75% value (Q3), where the middle half of the data lives. Anything more than **1.5 × IQR** below Q1 or above Q3 is flagged. `quantile(0.25)` gives Q1:

```
q1 = students["study_hours"].quantile(0.25)
q3 = students["study_hours"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
print(f"Q1 = {q1:.2f}, Q3 = {q3:.2f}, IQR = {iqr:.2f}")
print(f"Normal range: {lower:.2f} to {upper:.2f} hours")

too_low = students["study_hours"] < lower
too_high = students["study_hours"] > upper
outliers = students[too_low | too_high]
outliers
```

```
Q1 = 6.85, Q3 = 11.53, IQR = 4.67
Normal range: -0.16 to 18.54 hours
```

	school	study_hours	sleep_hours	attendance	score	passed
17	North	38.0	7.7	91.0	59.0	False
52	East	19.2	8.1	89.0	87.0	True
88	North	42.5	6.9	85.0	44.0	False
140	South	45.0	6.5	93.0	44.0	False

Four students are flagged. 38, 42.5 and 45 hours a week of study, with fairly low scores, look like typing mistakes. But 19.2 hours is possible for a hard-working student. The rule only **flags** candidates; you decide what to do with each. Let's see how much the three mistakes were hiding:

```
typos = students["study_hours"] > 30
fixed = students[~typos]      # every student except the three typos

r_before = students["study_hours"].corr(students["score"])
r_after = fixed["study_hours"].corr(fixed["score"])
print(f"r with the typos:    {r_before:.2f}")
print(f"r without the typos: {r_after:.2f}")
```

```
r with the typos:    0.35
r without the typos: 0.69
```

Three wrong values out of 200 almost **halved** the correlation. That's why outliers deserve attention before you draw any conclusion.

📝 Step 8: write down your findings

An analysis isn't finished until someone else can understand it. Write a few short sentences in plain words, with the numbers that support them, and mention the problems you found. f-strings (Day 2) keep the numbers in the sentences up to date:

```
pass_rate = students["passed"].mean()   # True counts as 1: the mean is the share
best_school = by_school["score"].idxmax()

print(f"Finding 1: More study goes with higher scores (r = {r_after:.2f}).")
print(f"Finding 2: {pass_rate:.0%} passed. {best_school} has the best average,")
print("           but the schools differ by only a few points.")
print("Finding 3: 9 sleep values are missing; 3 study times look like typos.")
```

```
Finding 1: More study goes with higher scores (r = 0.69).
Finding 2: 57% passed. North has the best average,
           but the schools differ by only a few points.
Finding 3: 9 sleep values are missing; 3 study times look like typos.
```

`{pass_rate:.0%}` shows a share as a percentage with 0 decimals (0.57 becomes 57%). Good findings are short, honest about uncertainty, and never claim more than the data shows ("goes with", not "causes").

⚠️ Common mistakes

- Jumping into charts with no question in mind. Write your questions first.
- Calling `.corr()` on a table with text columns → `ValueError: could not convert string to float`. Select the number columns.
- Reading correlation as cause: "goes together with" is all r can say.

- Deleting every flagged outlier automatically. Check each one: is it a mistake or a rare but real value?
- Trusting small differences between groups. A few points with small groups may just be luck.

✅ Recap

- The EDA routine: questions → shape/types → missing → distributions → groups → relationships → outliers → findings.
- `describe()` gives count, mean, std, min, quartiles and max; check min and max for odd values.
- `value_counts(normalize=True)` gives shares; `pd.crosstab(a, b, normalize="index")` compares two categories.
- `df[cols].corr()` and `a.corr(b)` give r between –1 and +1. Correlation is not causation.
- IQR rule: `iqr = q3 - q1`; flag values below `q1 - 1.5*iqr` or above `q3 + 1.5*iqr`.
- Finish with a few plain-English findings, numbers included.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Missing values report

Create three variables:

- `missing` : the number of empty cells in each column (a Series)
- `most_missing` : the **name** of the column with the most empty cells, found with code
- `missing_pct` : that column's empty cells as a **percentage** of all rows

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

# Create missing, most_missing and missing_pct below
```

Hints:

1. `missing = students.isna().sum()`
2. `.idxmax()` gives the label (here: the column name) of the biggest value.
3. `missing_pct = missing.max() / len(students) * 100`

2. Compare the schools

Use `groupby` with named aggregations (Day 11) to build a table `school_summary` with one row per school and two columns: `mean_score` (average score) and `mean_sleep` (average sleep hours). Then store the difference between the highest and the lowest average score in `gap`.

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

# Create school_summary and gap below
```

Hints:

1. Named aggregation: `new_name=("column", "mean")` inside `.agg(...)`.
2. `school_summary = students.groupby("school").agg(mean_score=("score", "mean"), mean_sleep=("sleep_hours", "mean"))`
3. `gap = school_summary["mean_score"].max() - school_summary["mean_score"].min()`

3. Sleep or study?

Calculate the correlation of `sleep_hours` with `score` in `r_sleep`, and of `attendance` with `score` in `r_attendance`. Then use an `if` to store the name of the column with the **stronger** (bigger) r in `stronger`: either `"sleep_hours"` or `"attendance"`.

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

# Create r_sleep, r_attendance and stronger below
```

Hints:

1. `r_sleep = students["sleep_hours"].corr(students["score"])` (empty cells are skipped automatically).
2. Do the same for `attendance`.
3. `if r_sleep > r_attendance:` then `stronger = "sleep_hours"`, `else:` `stronger = "attendance"`.

4. Outliers in attendance

Use the IQR rule on the `attendance` column. Create `q1`, `q3`, `lower` and `upper` (the limits Q1 − 1.5 × IQR and Q3 + 1.5 × IQR), and store **how many students** are outside the limits in `n_outliers`.

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

# Create q1, q3, lower, upper and n_outliers below
```

Hints:

1. `q1 = students["attendance"].quantile(0.25)`, and `0.75` for q3.
2. `iqr = q3 - q1`, `lower = q1 - 1.5 * iqr`, `upper = q3 + 1.5 * iqr`
3. Filter with `|` as in the lesson, then count the rows: `n_outliers = len(...)`.

5. Challenge: three findings

Answer three EDA questions with code, then report them:

- `pass_rate`: the share of students (between 0 and 1) with a score of **60 or more**
- `best_school`: the school with the highest **pass rate** (not the highest average!)
- `r_study`: the correlation of study hours with score, **after removing** students who study more than 30 hours a week

Finally, print three lines that start with `Finding 1:`, `Finding 2:` and `Finding 3:`, each one using one of the variables in an f-string.

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

# 1. pass_rate
# 2. best_school
# 3. r_study
# Then print Finding 1, 2 and 3
```

Hints:

1. Make a True/False column: `students["passed"] = students["score"] >= 60`. The mean of True/False values is the share of Trues.
2. `best_school = students.groupby("school")["passed"].mean().idxmax()`
3. `ok = students[students["study_hours"] <= 30]`, then `r_study = ok["study_hours"].corr(ok["score"])`. Print with `print(f"Finding 1: {pass_rate:.0%} of students passed.")`

➡️ Tomorrow

Mini-project: ship passengers EDA. You'll explore the passenger list of the made-up ship *Northern Star* (inspired by the famous Titanic dataset) from start to finish: clean it, compare survival rates by sex, class and age, draw the charts and write your own findings.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Missing values report

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

missing = students.isna().sum()
most_missing = missing.idxmax()
missing_pct = missing.max() / len(students) * 100
print(missing)
print(f"{most_missing}: {missing_pct:.1f}% missing")
```

2. Compare the schools

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

school_summary = students.groupby("school").agg(
    mean_score=("score", "mean"),
    mean_sleep=("sleep_hours", "mean"),
)
gap = school_summary["mean_score"].max() - school_summary["mean_score"].min()
print(school_summary.round(2))
print(f"Gap: {gap:.2f} points")
```

3. Sleep or study?

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

r_sleep = students["sleep_hours"].corr(students["score"])
r_attendance = students["attendance"].corr(students["score"])
if r_sleep > r_attendance:
    stronger = "sleep_hours"
else:
    stronger = "attendance"
print(f"Sleep: r = {r_sleep:.2f}, attendance: r = {r_attendance:.2f}")
print("Stronger:", stronger)
```

4. Outliers in attendance

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

q1 = students["attendance"].quantile(0.25)
q3 = students["attendance"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
too_low = students["attendance"] < lower
too_high = students["attendance"] > upper
n_outliers = len(students[too_low | too_high])
print(f"Normal range: {lower} to {upper}")
print("Outliers:", n_outliers)
```

5. Challenge: three findings

```
import numpy as np
import pandas as pd

# Made-up survey of 200 students (same as the lesson)
rng = np.random.default_rng(42)
n = 200
school = rng.choice(["North", "South", "East"], size=n, p=[0.4, 0.35, 0.25])
study_hours = rng.normal(9, 3.5, n).clip(0, 20).round(1)
sleep_hours = rng.normal(7, 1, n).clip(4, 10).round(1)
attendance = rng.normal(88, 7, n).clip(55, 100).round()
noise = rng.normal(0, 8, n)
score = (-8 + 3 * study_hours + 2 * sleep_hours + 0.3 * attendance
         + 6 * (school == "North") + noise).clip(0, 100).round()
students = pd.DataFrame({"school": school, "study_hours": study_hours,
                         "sleep_hours": sleep_hours, "attendance": attendance,
                         "score": score})
students.loc[rng.choice(n, size=9, replace=False), "sleep_hours"] = np.nan
students.loc[[17, 88, 140], "study_hours"] = [38.0, 42.5, 45.0]

students["passed"] = students["score"] >= 60
pass_rate = students["passed"].mean()
best_school = students.groupby("school")["passed"].mean().idxmax()

ok = students[students["study_hours"] <= 30]
r_study = ok["study_hours"].corr(ok["score"])

print(f"Finding 1: {pass_rate:.0%} of students scored 60 or more.")
print(f"Finding 2: {best_school} has the highest pass rate.")
print(f"Finding 3: Study time goes with higher scores (r = {r_study:.2f}).")
```